

06

Software Maintenance

Continuous improvements, security updates, optimization and technical support for existing software.

Project Overview

A business was running a PHP application built years earlier by a developer who had since moved on. It still worked, but it was slow, out of date and nobody was confident enough to change anything. We took over long-term maintenance and gradually modernised it.

The Challenge

- Outdated framework and dependency versions with known security issues.
- No version control history, backups or documentation to work from.
- Slow pages caused by unindexed tables and repeated database queries.
- Every small change risked breaking something else, so improvements had stalled.

Our Solution

- Placed the entire codebase under Git version control and documented the current architecture.
- Upgraded PHP and Laravel dependencies in staged, tested increments rather than one risky jump.
- Patched security issues, hardened input validation and tightened access rules.
- Profiled and optimised slow MySQL queries, adding proper indexes and eager loading.
- Established automated backups, error monitoring and a staging environment for safe testing.
- Set up a monthly maintenance cycle with a clear support channel and response commitment.

Technologies Used

Frontend	HTML5, CSS3, JavaScript, Bootstrap, Vue.js
Backend	PHP, Laravel, REST API
Database	MySQL
Dev & Collaboration	Git, GitHub, VS Code, XAMPP

Results

Metric	Before	After
Known security vulnerabilities	Several open	Resolved and monitored
Slowest page load	approx. 6 s	approx. 1.4 s
Backups	None	Automated daily
Unplanned downtime	Occasional	Minimal, with monitoring

Key Takeaway

Maintenance is not just fixing what breaks. Version control, monitoring, backups and small incremental upgrades turn a fragile legacy system into one the business can keep investing in with confidence.

Note: Figures shown are representative project outcomes and can be replaced with your own verified client data before sharing.